

46-927, Spring 2021: Homework #5

Due 1:30pm, Wednesday, March 3, 2021.

For this homework, we're back to Python. (Whew.)

We are also entering the part of class where the homeworks will be getting shorter and simpler, to give you more time with your projects. This homework is a bit simpler than they have been in the past, focusing on giving you a bit of a feel for PCA/MDS/isomap through simple examples.

1. We are going to use the digits data, similar to the last homework, to visually explore the meaning of principal components. For this homework, we will focus only on “3” digits, since they have an interesting variety of shapes. You can find 658 threes in `threes.csv`. The format is like last week's homework: each row corresponds to a 16×16 digit, unraveled into a vector. Each pixel takes values between 0 and 1 (unlike last week, where they were actually in $[-1, 1]$).

You will also find a `plot_digit.py` with Python code for plotting a digit row, similar to the R code we used last homework.

- (a) Compute the principal component directions and principal component scores of `threes`. Plot the first two principal component scores (the x -axis being the first score and the y -axis being the second score). Note that each point in this plot corresponds to an image of a “3”.

Hints:

To compute PCA, you can use the `PCA` function from `sklearn.decomposition`. It is called similarly to other `sklearn` functions. The only catch is that it is implemented as a transformation. You need to fit the model, and then call `transform` to get the PCA scores. You can also use the `fit_transform` function to do both at the same time.

The numbers returned by `transform` or `fit_transform` are your scores. The components (your V) is in `pca.components_` (if `pca` is your fitted object).

- (b) Let's look at the first two principal component scores more closely. One way to do this is to sample a representative grid of points, and then plot the corresponding digits. You will do this by looking at the 5%, 25%, 50%, 75%, 95%

percentile points of each of the first two scores, and finding the closest points to their cross-product (for example, the digit closest to the 5th percentile of score 1 and the 75th percentile of score 2).

Start by just drawing the horizontal and vertical lines corresponding to these percentiles on top of your scatter plot from the last part. You'll be plotting the digits everywhere these lines cross. (Use `np.percentile` to compute percentiles of each score, noting that this function takes numbers in $[0, 100]$ instead of $[0, 1]$.)

I've given you a helper function in `quantile_helper.py` to find the indices near each of these crossing points. You just feed it your two columns of scores, and the vector of percentiles of interest (again in $[0, 100]$).

Make a 5×5 grid of the digit plots corresponding to these intersections.

- (c) Looking at these digits, what can be said about the nature of the first two principal component scores? (The first principal component score is increasing as you move from left-to-right in any of the rows. The second principal component score is decreasing as you move from top-to-bottom in any of the columns.) In other words, I'm asking you to explain what changes with respect to changes in each of the component scores.
- (d) Plot the proportion of variance explained by the first k principal component directions, as a function of $k = 1, \dots, 256$. How many principal component directions would we need to explain 50% of the variance? How many to explain 90% of the variance?
(See `pca.explained_variance` and `pca.explained_variance_ratio`.)
- (e) Finally, plot the first four principal component directions. You'll find these in `pca.components_`, which is (number of components) $\times$ (number of pixels). Plot these vectors like 16×16 digits, using the same function as you do for digits. Make sure it is clear which plot is which component. You should interpret these directions as "changes" from a mean three.

Can you see a correspondence between the pattern in your grid from (b) and (c), and these direction vectors?

2. *Unfolding a spiral.* This problem uses the data in `spiral.csv` from Canvas. This contains columns `x1`, `x2`, and `y`.

- (a) The data set consists of two covariates, `x1` and `x2`, along with an outcome `y` of interest. Begin by plotting the data. Plot x_1 versus x_2 . Color the points by their `y` values, so you can see if there is a pattern. What do you see? Suppose

we use linear regression; does it seem like x_1 and x_2 will work well for predicting y ?

Hint: You can use `plt.scatter` to make the plot, with the `c` argument setting the color to the y vector.

- (b) Let's actually try it! Regress y on x_1 and x_2 . What does your training error MSE look like? (We're just messing around, don't worry about a test set for this problem.) Plot your estimated $\hat{y}$ versus y . Are we doing well?

- (c) First, compute the usual Euclidean distance between all your points. You may want to use `sklearn.metrics.pairwise.euclidean_distances`. To see that these distances are odd, make a plot of the distance from every point to the 25th point (middle point). (That is, plot the 25th row of your distance matrix, remembering that python is 0 indexed.) Note that the Euclidean distances oscillate instead of increasing monotonically away from the 25th location. This happens because they are not distances along the spiral.

- (d) Now we will try isomap to see whether we can find a better space for visualizing the data and predicting. For this problem, we will use the `Isomap` function from `sklearn.manifold`.

Using this function, fit Isomap to x_1 and x_2 . Use `n_neighbors = 1` and `n_components = 2`.

First, let's look at the new isomap distance. You can find this distance matrix in `iso.dist_matrix_`, if `iso` is the fitted isomap object. Again, plot the distance from every point to the 25th point. Does this distance increase monotonically away from the 25th point?

Finally, plot the points in the new isomap coordinates. To get these coordinates, you will use `isomap.transform` on your data. Remake your scatterplot from the first part of this problem, now in these new coordinates. Keep the colors from y . What do you see?

- (e) Try using the new coordinates for prediction! Regress y on Z . What is your training error? How does it compare to (a)?